Real-Time Stage-Level Evaluation of SIMD-Native Chaos-Based Image Encryption Pipelines

Ravikanth Reddy Gudipati[1*]

[1*]University at Buffalo, Buffalo, NY, USA.

Corresponding author(s). E-mail(s): ravikanthreddy@ieee.org;

Abstract

Chaos-based image-encryption studies commonly report aggregate running time and image-domain statistics, but often do not show whether the proposed pipeline can satisfy real-time image-processing budgets or which stage prevents it from doing so. This work presents a reproducible C++17 benchmark that decomposes representative chaotic image-encryption methods into keystream generation, pixel permutation, and diffusion, then evaluates end-to-end latency, throughput, and stage share against frame-rate-oriented constraints. Traditional components are compared with SIMD-oriented replacements on 12 deterministic synthetic images, the 24-image Kodak PhotoCD dataset, and a public 720p/1080p video-frame workload. The still-image evaluation contains 14,730 performance observations and 3,300 full-scheme security observations, with ten repetitions per configuration; the real-time extension adds matched x86-64 and ARM64 video-frame measurements with mean latency, p95 latency, FPS-equivalent throughput, and 30/60 FPS feasibility labels. Replacing chaotic sort permutation with a linear block permutation improves mean permutation throughput by 100.6–131.9 times. Replacing a global feedback chain with AVX2 multi-lane or tree diffusion improves mean diffusion throughput by approximately 1.8–2.1 times. The fastest benchmark-informed candidate, Checkerboard-CA-ARX, sustains 259.1 MiB/s on Kodak images and 200.0 MiB/s at 4096×4096, while ChaCha20 remains substantially faster. Security experiments show that favorable entropy, histogram, correlation, and key-sensitivity values do not imply resistance to differential or known-plaintext attacks. The candidate pipelines are therefore presented as real-time performance templates rather than deployment-ready ciphers.

Keywords: chaotic image encryption, real-time image processing, SIMD, stage-level benchmarking, permutation-diffusion, image cryptanalysis, latency

1 Introduction

Real-time image-processing systems need predictable latency, not only visually plausible ciphertexts or high aggregate throughput. A 30-frame/s pipeline has about 33.3 ms to process each frame, while a 60-frame/s pipeline has about 16.7 ms before accounting for capture, transport, display, storage, or downstream analytics. Chaos-based image-encryption papers often report image-domain statistics and a single running time, but that evidence rarely shows whether a design can satisfy these frame budgets on commodity CPU architectures.

The permutation-diffusion architecture is a recurring structure in chaos-based image encryption. A permutation stage rearranges pixel positions, and a diffusion stage modifies pixel values using a key-dependent sequence. The architecture maps naturally to common image statistics: permutation reduces adjacent-pixel correlation, while diffusion can flatten histograms and increase sensitivity to key changes.

However, three problems complicate real-time and security claims in this literature. First, complete-scheme timing hides the contribution of individual stages. A proposed SIMD diffusion kernel may be fast while an $O(N \log N)$ chaotic sort still dominates total execution time. Conversely, a sequential chaotic generator may prevent a pipeline from benefiting from an otherwise parallel permutation. Second, throughput alone can mask missed frame deadlines: a high mean MiB/s value is less useful without per-frame latency, p95 latency on frame sequences, and pass/fail status against 30/60 FPS budgets. Third, image-domain metrics such as entropy, correlation, NPCR, and UACI are diagnostic measurements rather than proofs of cryptographic security. A deterministic stream-XOR construction can produce a uniform-looking ciphertext while remaining vulnerable when a key/nonce pair is reused.

This study investigates how implementation-friendly stages affect the performance of chaotic image-encryption methods without conflating that engineering result with cryptographic security. Traditional chaotic components, SIMD-oriented alternatives, and standard cryptographic baselines are implemented within one benchmark suite. The experiments isolate keystream, permutation, and diffusion execution time before composing selected components into benchmark-informed candidate pipelines.

The study answers five research questions:

- **RQ1:** Which stages dominate representative chaotic image-encryption pipelines?
- **RQ2:** Which replaceable stage designs benefit most from linear-time and SIMD-oriented restructuring?
- **RQ3:** Do stage-level improvements remain visible in composed candidate pipelines and at larger image sizes?
- **RQ4:** Which candidates meet 30-frame/s and 60-frame/s latency budgets at common image and video-frame resolutions on measured CPU architectures?
- **RQ5:** Which security conclusions are, and are not, supported by common image-domain metrics?

The contributions are:

1. A reproducible C++17/OpenCV/OpenSSL benchmark with a common cipher interface and separate keystream, permutation, diffusion, and total timing.

2. A real-time evaluation protocol that reports milliseconds per frame, FPS-equivalent throughput, stage shares, and pass/fail status against 30/60 FPS budgets.
3. A controlled comparison of traditional components and SIMD-oriented replacements across synthetic images, real natural images, video frames, and resolutions up to 4096×4096.
4. Evidence that linear block permutation and lane/tree diffusion remove major implementation bottlenecks, while scalar floating-point maps and sort-based permutation remain expensive.
5. A negative security result demonstrating that strong image statistics can coexist with differential and reused-nonce known-plaintext weaknesses.
6. An explicit boundary between fast image-domain transformation prototypes and cryptographically deployable encryption.

2 Background and Related Work

2.1 Permutation-Diffusion Image Encryption

Shannon identified confusion and diffusion as fundamental properties of secret-key systems [1]. Fridrich adapted related ideas to image encryption using two-dimensional chaotic maps and a permutation-diffusion architecture [2]. Many later schemes use pixel-, bit-, or block-level permutation followed by one or more diffusion rounds [3, 4].

Traditional chaotic permutation often generates one chaotic score per pixel and sorts pixel indices by those scores. The method is simple to describe but requires $O(N \log N)$ comparisons and irregular memory access. Diffusion is frequently implemented as the global recurrence

$$C[i] = P[i] \oplus K[i] \oplus C[i-1], \tag{1}$$

where P, K, and C denote permuted plaintext, keystream, and ciphertext. This construction creates an apparent avalanche path but also introduces a loop-carried dependency that restricts vectorization.

2.2 Chaotic and Spatial Keystream Generators

Scalar logistic, tent, and sine maps are common keystream sources. Their iterative dependency and floating-point operations can limit throughput. Spatial systems such as coupled-map lattices and cellular automata expose multiple local states and are therefore more compatible with lane-level or data-parallel execution [5]. This work evaluates both categories, as well as Hamiltonian-inspired and reaction-diffusion generators.

2.3 SIMD-Oriented Restructuring

Single Instruction, Multiple Data (SIMD) instructions operate on multiple values per instruction. On the evaluated x86-64 host, AVX2 provides 256-bit integer vectors. SIMD benefits are largest when work can be expressed as regular independent operations. Sorting, pointer chasing, and global recurrences are less suitable.

Associative operations can sometimes be reformulated as scans. The global XOR chain can be written as

$$X[i] = P[i] \oplus K[i], \tag{2}$$
$$C[i] = IV \oplus X[0] \oplus \cdots \oplus X[i]. \tag{3}$$

This permits block-wise prefix processing with a carry between blocks. Multi-lane chaining instead maintains 32 independent byte chains per AVX2 vector:

$$C[i] = P[i] \oplus K[i] \oplus C[i - 32]. \tag{4}$$

Tree diffusion applies shuffle-and-XOR stages to mix bytes within a vector in logarithmic depth. Parallel scan and SIMD prefix techniques are established general-purpose primitives [6].

2.4 Security Evaluation

Entropy, histogram uniformity, adjacent-pixel correlation, NPCR, and UACI are widely reported in image-encryption studies. NPCR and UACI quantify output changes caused by a small input modification [7]. These metrics are useful, but they do not replace cryptanalysis. Prior work has demonstrated chosen-plaintext and structural attacks against permutation-diffusion image ciphers [8, 13].

The present study therefore includes AES-256-CTR and ChaCha20 as standardized or widely deployed cryptographic reference points [14, 15]. BLAKE3 keyed extendable output is also evaluated as a high-quality keystream source [16]. These baselines anchor the performance discussion; the proposed stage combinations are not claimed to provide equivalent security.

3 Benchmark Design

3.1 Common Pipeline Model

Each complete scheme implements a common *IImageCipher* interface. Encryption is represented as three timed stages:

1. **Keystream generation:** produce N key-dependent bytes.
2. **Permutation:** rearrange pixels, bytes, blocks, or local positions.
3. **Diffusion:** combine the permuted data and keystream.

Total execution time includes stage execution and required data conversion or allocation overhead. Throughput, reported in mebibytes per second, is

$$\text{throughput} = \frac{\text{image bytes}}{\text{total execution time}}. \tag{5}$$

The suite also executes each replaceable stage independently. Checksums prevent unused outputs from being optimized away.

3.2 Full-Scheme Baselines

The full-scheme suite contains scalar chaotic XOR methods using logistic, tent, sine, coupled-lattice, and Hamiltonian-lattice generators; permutation plus XOR methods using logistic sort, Arnold map, and tiled Arnold map; an official BLAKE3 keyed-XOF keystream followed by XOR; and AES-256-CTR and ChaCha20 through OpenSSL EVP.

All full schemes are reversible and checked by encryption followed by decryption. The benchmark intentionally reuses a fixed key and nonce across security probes to expose reused-keystream behavior. This is a misuse test, not a recommended operating mode for AES-CTR, ChaCha20, or BLAKE3.

3.3 Replaceable Stages

The isolated keystream experiments evaluate scalar logistic and sine maps, fixed-point tent, a 16-state coupled-map lattice (CML), a 64-cell rule-30-like cellular automaton (CA), a Hamiltonian-inspired lattice, and a reaction-diffusion generator. These generators compare computational structures and are not claimed to provide cryptographic unpredictability.

The permutation experiments compare chaotic score sort, sequential random walk, Arnold/cat map, affine modular mapping, key-derived Feistel-like mapping with cycle walking, 256-byte block permutation, and four rounds of disjoint checkerboard swaps. The block and checkerboard implementations are deliberately simple performance templates. They are fixed rather than key-derived and must not be interpreted as secure permutations.

The diffusion experiments include global and block-local XOR chains, blocked prefix processing, a two-dimensional stencil, ARX block and bit-plane-style transforms, forward and reverse prefix XOR, 32-lane AVX2 chaining, AVX2 tree XOR, a forward-prefix/tree/reverse-prefix composition, and ARX prefix modulo 256. The prefix-XOR prototype uses AVX2 loads, XORs, broadcasts, and stores, but its within-vector prefix helper performs a scalar scan. The multi-lane and tree variants contain the clearest AVX2-native data paths.

3.4 Candidate Pipelines

Seven candidate combinations test whether isolated improvements survive composition. Table 1 summarizes their stages.

These pipelines currently produce checksums for performance validation but do not implement complete decryptors or equivalent security probes. They are component-composition experiments, not proposed secure ciphers.

Table 1 Benchmark-Informed Candidate Pipelines

Candidate	Keystream	Permutation	Diffusion
CA-Feistel-ARX	Cellular automata	Feistel-like index	ARX block
Checkerboard-CA-ARX	Cellular automata	Checkerboard swaps	ARX block
Affine-CA-PrefixTree	Cellular automata	Affine mapping	Prefix/tree/reverse
Checkerboard-CA-MultilaneTree	Cellular automata	Checkerboard swaps	Multi-lane/tree
CML-Feistel-Stencil	CML	Feistel-like index	Stencil
Hamiltonian-Block-Stencil	Hamiltonian lattice	Block-Feistel mapping	Stencil
Affine-CML-Bitplane	CML	Affine mapping	Bit-plane transform

4 Experimental Methodology

4.1 Implementation and Platform

4.2 Use of Generative AI Assistance

The study used generative-AI and large-language-model assistance as an engineering and editorial aid during manuscript preparation. The assistance was limited to drafting and reorganizing prose, identifying claim-boundary issues, suggesting experiment-reporting checklists, helping convert the manuscript to the Springer Nature LaTeX template, and proposing small automation changes for dataset and result aggregation scripts. The benchmark design, source-code changes, experiment execution, result selection, numerical interpretation, and final claims were reviewed and accepted by the author. No generative-AI system was treated as an experimental instrument, no AI-generated numerical result was reported without verification from the committed CSV outputs, and no external source was cited solely from AI-generated text. The author takes full responsibility for the accuracy, integrity, and interpretation of the work.

The benchmark is implemented in C++17 and built with CMake in Release mode using GCC 11.4.0, `-O3`, `-march=native`, and explicit AVX2 support. OpenCV is used for image loading and output, OpenSSL EVP provides AES-256-CTR and ChaCha20, and the official BLAKE3 C implementation provides keyed XOF output.

Experiments ran on a four-core Intel Xeon E5-2620 v4 at 2.10 GHz under 64-bit Linux 5.15. The processor supports SSE2, AVX, AVX2, and AES instructions. BLAKE3 was compiled in portable mode, so its result does not include optional architecture-specific SIMD dispatch. Candidate and stage measurements are single-process measurements; this paper does not claim multicore scaling.

For a Journal of Real-Time Image Processing submission, the same harness is reported on two CPU architectures: an x86-64 SIMD host and an ARM64 host. The preferred final matrix can add a third modern x86 host so that the current older Xeon result is not the only x86 data point. Each platform report should include processor

model, core or vCPU count, memory, operating system, compiler version, compiler flags, SIMD capability, warm-up count, timed repetitions, and whether CPU frequency or affinity was controlled.

4.3 Datasets

Two dataset classes are used in the current results:

- **Synthetic controls:** deterministic gradient, texture, and noise images at 512×512, 1024×1024, 2048×2048, and 4096×4096.
- **Natural images:** all 24 color images from the Kodak PhotoCD dataset [17], comprising 768×512 and 512×768 orientations.

Synthetic images provide controlled size and content variation. Kodak images provide natural-image diversity while retaining a stable public benchmark. The Lena/Lenna image is explicitly excluded from the publication dataset and does not appear in the active benchmark results.

The real-time extension adds public video frames sampled at 1280×720 and 1920×1080 so that latency variation can be reported over frame sequences rather than still images alone. One literature-comparable image-encryption set, such as USC-SIPI with Lena/Lenna excluded, can be added only to connect the results to prior image-encryption studies. Domain datasets such as medical, surveillance, satellite, or industrial imagery should be added only if the manuscript claims that deployment domain.

4.4 Experiment Matrix

Every executed configuration is repeated ten times. Slow full schemes and intentionally slow stage baselines are limited to images no larger than 1024×1024. Fast stage alternatives continue to 4096×4096. Slow candidate pipelines continue to 2048×2048, and fast candidates continue to 4096×4096.

The final dataset contains 3,300 full-scheme performance records, 3,300 full-scheme security records, 9,000 isolated-stage performance records, and 2,430 candidate-pipeline performance records. Reported aggregate throughput values are arithmetic means. The 95% confidence interval is

$$\mathrm{CI}_{95} = \frac{1.96s}{\sqrt{n}}, \tag{6}$$

where s is the sample standard deviation. No outlier removal is applied.

4.5 Real-Time Metrics

The primary real-time unit is milliseconds per frame. For each scheme, dataset, resolution, and architecture, the benchmark should report mean latency, 95% confidence interval, FPS-equivalent throughput, and stage share for keystream generation, permutation, diffusion, and total execution. For video-frame workloads, the journal-ready

Table 2 Selected Aggregate Full-Scheme Throughput

Scheme	Image size	n	Mean MiB/s	95% CI	Speedup
ChaCha20	512×512	30	996.357	42.849	7.128×
ChaCha20	768×512	180	895.707	10.484	6.389×
ChaCha20	1024×1024	30	903.400	18.640	6.532×
AES-256-CTR	768×512	180	355.663	3.399	2.537×
AES-256-CTR	1024×1024	30	594.958	9.202	4.302×
BLAKE3-XOR	768×512	180	315.714	2.750	2.252×
BLAKE3-XOR	1024×1024	30	294.142	4.023	2.127×

table should also report p95 latency. FPS-equivalent throughput is computed as

$$\mathrm{FPS}_{eq} = \frac{1000}{\mathrm{ms/frame}}. \tag{7}$$

The real-time feasibility table should mark whether each configuration meets 30-frame/s and 60-frame/s budgets. The 30-frame/s budget is 33.3 ms/frame and the 60-frame/s budget is 16.7 ms/frame. These thresholds are intentionally reported as feasibility checks rather than deployment guarantees because a complete real-time system also includes acquisition, transport, display, storage, authentication, and application-specific processing.

4.6 Security Metrics

For full schemes, the suite records Shannon entropy, the 256-bin chi-square statistic, horizontal/vertical/diagonal adjacent-pixel correlation, NPCR and UACI after flipping one plaintext bit, NPCR after flipping one key bit, empirical known-plaintext and chosen-plaintext byte-recovery scores under a reused key/nonce, and exact decryption correctness.

For the byte-recovery probe, a keystream estimate is computed as $K'[i] = P_1[i] \oplus C_1[i]$ and applied to a second ciphertext. A score of 1.0 means complete recovery under this simple reused-keystream model.

5 Results

5.1 Full-Scheme Performance

Table 2 reports selected aggregate full-scheme results. ChaCha20 is the fastest complete baseline on all commonly measured image dimensions. AES-256-CTR is second among the standard baselines. BLAKE3-XOR is the fastest non-OpenSSL full scheme, but its portable-only BLAKE3 build makes this a conservative measurement.

The results do not support a claim that chaotic-image pipelines outperform standard ciphers. Instead, they show the cost of common chaotic components and provide a performance target for alternative stage designs.

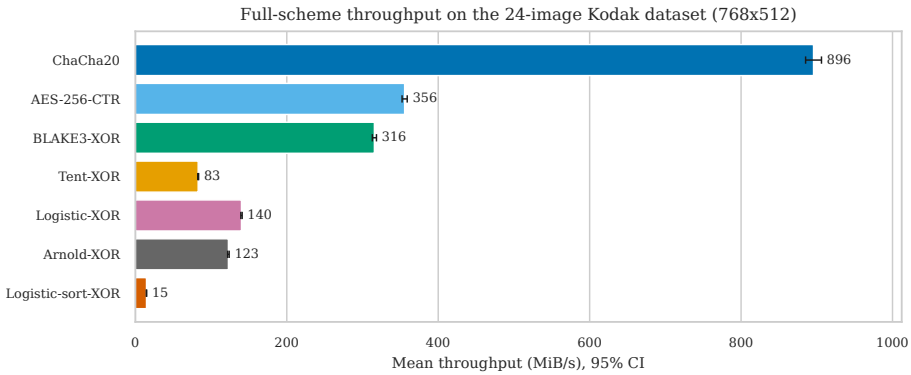

Fig. 1 Selected full-scheme throughput on the 24-image Kodak dataset. Error bars show 95% confidence intervals. Standard cryptographic baselines remain substantially faster than the evaluated chaotic full schemes.

Table 3 Block-Permutation Performance Relative to Chaotic Sort

Size	n	MiB/s	CI	Speedup
512×512	30	512.371	6.473	100.634×
512×768	60	499.309	4.729	110.013×
768×512	180	499.812	2.866	109.371×
1024×1024	30	497.839	5.289	131.861×

5.2 Isolated Permutation Performance

Permutation produces the largest stage-level improvement. At 512×512, block permutation reaches 512.371 MiB/s and is 100.634 times faster than chaotic sort. At 1024×1024 it reaches 497.839 MiB/s and is 131.861 times faster.

This result answers RQ1 and RQ2: chaotic sort is implementation-hostile, and replacing it with a regular linear-time transform changes the performance regime. The benchmark block permutation is fixed and must be made key-derived and cryptographically analyzed before use in a cipher.

5.3 Isolated Diffusion Performance

The fastest diffusion alternatives are multi-lane chaining and tree XOR. Their mean speedups over the global chain are approximately twofold.

Removing a global byte-to-byte dependency creates a measurable SIMD opportunity. It also explains why adding SIMD only to XOR inside a traditional end-to-end pipeline may show little total improvement: the remaining generator and permutation costs dominate.

Table 4 Selected Aggregate Diffusion Performance

Stage	Image size	n	Mean MiB/s	95% CI	Speedup
Multi-lane chain AVX2	512×512	30	499.419	7.774	2.076×
Multi-lane chain AVX2	768×512	180	499.120	3.046	2.095×
Multi-lane chain AVX2	1024×1024	30	493.067	8.000	2.054×
Tree XOR AVX2	512×512	30	470.202	7.667	1.955×
Tree XOR AVX2	768×512	180	469.901	2.793	1.973×
Tree XOR AVX2	1024×1024	30	464.386	8.035	1.935×
ARX block diffusion	768×512	180	426.568	2.540	1.791×

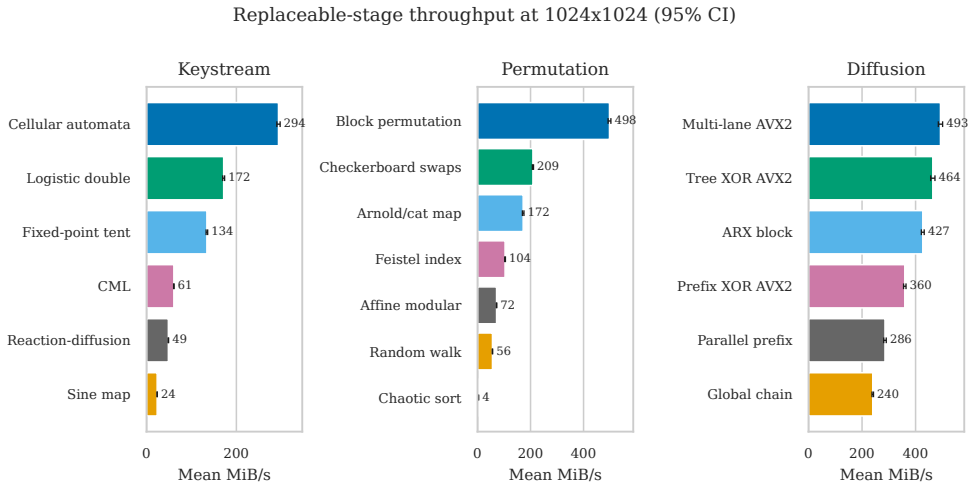

Fig. 2 Representative replaceable-stage throughput at 1024×1024. The largest gain is algorithmic: replacing chaotic sort with a regular linear-time permutation. AVX2-native multi-lane and tree diffusion also outperform the sequential global chain.

5.4 Candidate Pipeline Performance

Checkerboard-CA-ARX is the fastest composed candidate. On Kodak images it achieves 259.138 MiB/s, while Checkerboard-CA-MultilaneTree achieves 232.142 MiB/s. Both remain slower than AES-256-CTR and ChaCha20 on the same image dimensions.

Table 6 shows that the fast checkerboard candidates distribute work across all three stages, whereas Feistel indexing dominates CA-Feistel-ARX and Hamiltonian generation dominates Hamiltonian-Block-Stencil. These results support RQ3: isolated-stage choices remain visible after composition.

5.5 Video-Frame Real-Time Feasibility

The real-time extension reruns the benchmark on a public video-frame workload at 1280×720 and 1920×1080. Each row in Table 7 summarizes 36 measurements for the

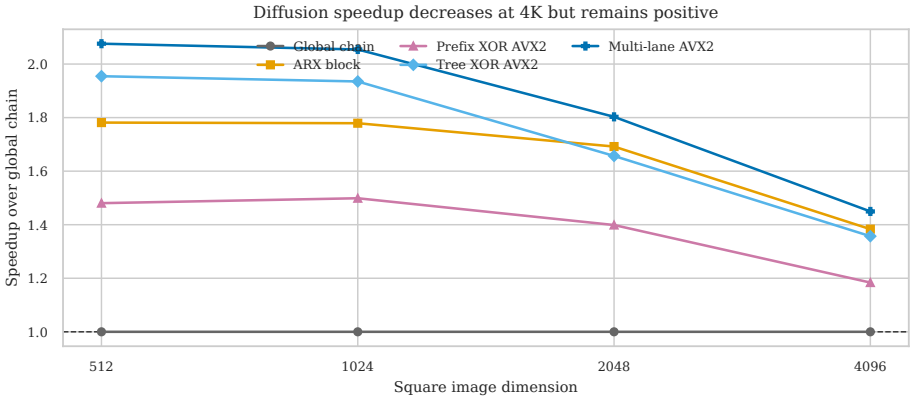

Fig. 3 Diffusion speedup relative to the global chain on square images. AVX2 multi-lane and tree diffusion retain a positive advantage at all measured sizes, although the advantage narrows at 4096×4096.

Table 5 Aggregate Candidate-Pipeline Throughput

Candidate	Image size	n	Mean MiB/s	95% CI
Checkerboard-CA-ARX	512×512	30	267.596	3.771
Checkerboard-CA-ARX	768×512	180	259.138	2.666
Checkerboard-CA-ARX	1024×1024	30	234.149	4.490
Checkerboard-CA-ARX	2048×2048	30	216.487	3.933
Checkerboard-CA-ARX	4096×4096	30	199.993	4.384
Checkerboard-CA-MultilaneTree	768×512	180	232.142	2.111
Checkerboard-CA-MultilaneTree	4096×4096	30	159.392	3.931

Table 6 Mean Candidate Stage Shares

Candidate	Key	Perm.	Diff.
Checkerboard-CA-ARX	47.69%	27.86%	24.44%
Checkerboard-CA-MultilaneTree	38.44%	22.60%	38.95%
CA-Feistel-ARX	11.69%	82.32%	5.99%
Hamiltonian-Block-Stencil	83.78%	11.97%	4.24%

corresponding scheme, resolution, and architecture. The x86-64 run used the local Xeon host; the ARM64 run used an AWS aarch64 host. The table is intentionally a feasibility check rather than a deployment guarantee because complete real-time systems must also budget capture, transport, display, storage, authentication, and downstream processing.

These results answer RQ4 for the measured hosts. The fastest chaotic candidate meets 30 FPS at both video resolutions on both architectures. It meets 60 FPS at 720p on x86-64 and at both resolutions on the measured ARM64 host, but it misses the 60

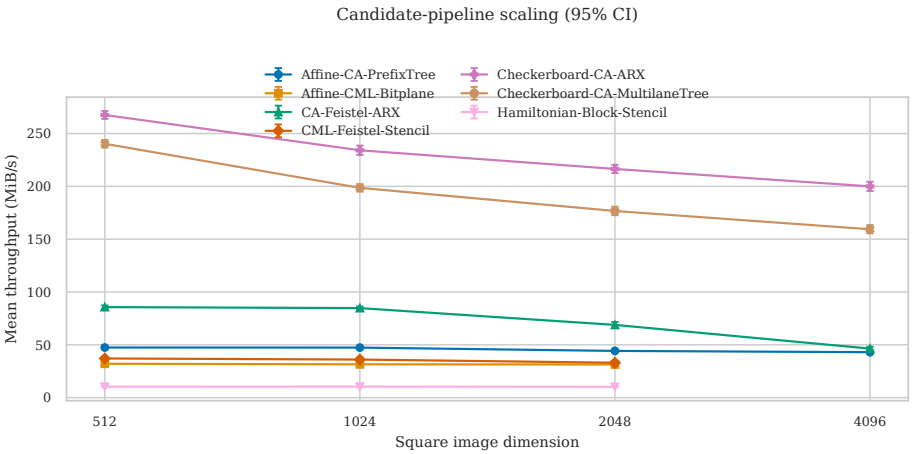

Fig. 4 Candidate-pipeline throughput across square image sizes. The checkerboard candidates remain fastest, but all candidates lose throughput as working sets grow. Error bars show 95% confidence intervals.

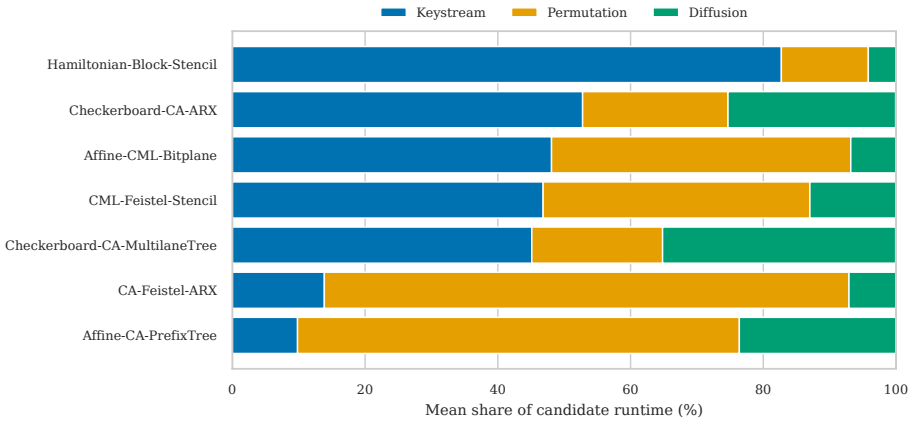

Fig. 5 Mean runtime composition of the candidate pipelines. The plot exposes the dominant bottleneck in each composition and shows why isolated stage measurements remain necessary.

FPS threshold at 1080p on the x86-64 host. The standardized cryptographic baselines remain faster and meet both frame-rate thresholds in all measured video-frame cases.

5.6 Cross-Architecture Reproducibility

To check whether the conclusions depend on one CPU family, the publication harness was rerun on the local Intel Xeon E5-2620 v4 host with AVX2 and on an AWS aarch64 host. The matched run used synthetic sizes 256, 512, and 1024, three repetitions, one warm-up, four timed iterations, and the Kodak pair `kodim01.png` and `kodim02.png`. The paired geometric-mean analysis over five images preserved the same qualitative

Table 7 Video-Frame Real-Time Feasibility on x86-64 and ARM64

Arch.	Scheme	Res.	Mean	p95	FPS_{eq}	30	60
x86-64	Ckbd-CA-ARX	1280×720	12.77	14.06	78.30	yes	yes
x86-64	Ckbd-CA-ARX	1920×1080	28.99	31.06	34.49	yes	no
x86-64	AES-GCM	1280×720	3.02	3.44	330.59	yes	yes
x86-64	AES-GCM	1920×1080	7.82	8.43	127.82	yes	yes
x86-64	ChaCha20	1280×720	3.28	3.56	304.72	yes	yes
x86-64	ChaCha20	1920×1080	8.44	9.39	118.55	yes	yes
ARM64	Ckbd-CA-ARX	1280×720	5.44	5.87	183.85	yes	yes
ARM64	Ckbd-CA-ARX	1920×1080	12.01	12.84	83.24	yes	yes
ARM64	AES-GCM	1280×720	1.88	2.06	531.20	yes	yes
ARM64	AES-GCM	1920×1080	4.06	4.12	246.59	yes	yes
ARM64	ChaCha20	1280×720	3.19	3.39	313.92	yes	yes
ARM64	ChaCha20	1920×1080	7.01	7.20	142.71	yes	yes

Ckbd-CA-ARX denotes Checkerboard-CA-ARX. Mean and p95 are in milliseconds per frame.

Table 8 Representative Cross-Architecture Paired Speedups

Case	x86	ARM
Map + scan exact	1.166× / 1.074×	1.305× / 1.103×
Symbolic + scan exact	1.153× / 1.053×	1.308× / 1.115×
Bitplane + scan exact	1.101× / 0.933×	1.249× / 1.068×
Sort logistic + scan exact	1.032× / 1.012×	1.042× / 1.016×

ordering on both systems: `scan_exact` consistently beat the scalar chain for the map and symbolic families, while `sort_logistic` remained near parity.

These paired runs support the claim that the main throughput ranking is not an artifact of the Xeon server alone. The exact-scan variant is the most robust replacement across both architectures, even though the absolute gains differ slightly by host and stage family.

5.7 Recent Literature Positioning

The recent chaos-based image-encryption literature still emphasizes confusion-diffusion compositions, hybrid key schedules, geometric block permutation, and statistical security diagnostics. Representative recent examples include multithreaded parallel confusion and diffusion for video frames [9], a medical-imagery permutation-diffusion design [10], geometric block permutation with dynamic substitution [11], and data-identified discrete chaotic maps [12].

This is not a substitute for a full peer-reviewed 2022–2026 survey matrix. It is the manuscript-level positioning needed to show that the present work is about measured implementation cost and reproducibility, not a new claim of cryptographic superiority. The companion notes retain the fuller matrix template for later journal-specific expansion.

Table 9 Representative Recent Works and the Remaining Measurement Gap

Year	Representative work	Family / emphasis	Gap relative to this paper
2023	[9]	Multithreaded parallel confusion-diffusion for video frames	Reports speed and statistical tests, but not stage-level attribution across architectures.
2024	[10]	Chaos-based permutation-diffusion for diagnostic imagery	Focuses on image statistics and functional encryption, not SIMD stage decomposition.
2025	[11]	Geometric block permutation with dynamic substitution	Emphasizes statistical security, not finite-precision or cross-CPU benchmarking.
2026	[12]	Data-identified discrete chaotic maps	Explores a newer map family, but still centers on cipher statistics rather than stage timing.

Table 10 Mean Security Diagnostics Across 300 Observations per Scheme

Scheme	Entropy	χ^2	Abs. corr.	NPCR	Key sens.	KPA
ChaCha20	7.999845	258.874	0.001216	0.000084%	99.619%	1.000
AES-256-CTR	7.999843	259.125	0.001148	0.000084%	99.611%	1.000
BLAKE3-XOR	7.999850	247.607	0.001419	0.000084%	99.607%	1.000
Tent-XOR	7.999828	282.731	0.001721	0.000084%	99.623%	1.000
Logistic-XOR	7.984321	26641.399	0.020440	0.000084%	99.387%	1.000
Coupled-lattice-XOR	7.971770	45857.479	0.137337	0.000084%	99.410%	1.000

5.8 Full-Scheme Security Diagnostics

All complete schemes decrypt correctly. Several also produce near-eight-bit entropy, low adjacent-pixel correlation, and strong key sensitivity. These positive metrics are insufficient to establish security.

The KPA recovery score of 1.0 for AES-CTR, ChaCha20, BLAKE3-XOR, and the stream-XOR chaotic variants is expected because the experiment deliberately reuses the same key/nonce. It demonstrates misuse sensitivity, not a break of the underlying standardized primitive. The negligible plaintext NPCR is also expected for CTR- or stream-XOR encryption: changing one plaintext bit changes only the corresponding ciphertext bit when the keystream is unchanged.

This result answers RQ5. Entropy, correlation, and key sensitivity can look excellent while a construction still lacks plaintext avalanche and fails under reused keystream. NPCR is not an appropriate requirement for all secure encryption modes;

Table 11 Negative-Control Outcomes

Control	Measured outcome	Interpretation
Reused key/nonce	KPA recovery score is 1.000 for ChaCha20, AES-CTR, BLAKE3-XOR, Tent-XOR, Logistic-XOR, and Coupled-lattice-XOR.	This is a misuse probe. It shows deterministic recovery under repeated keystream, not a break of the underlying primitive.
One-bit plaintext flip	NPCR is 0.000084% for the same full-scheme controls.	Stream and CTR-style constructions do not provide plaintext avalanche under keystream reuse; this is the expected control result.
One-bit key flip	Key sensitivity stays in the 99.387–99.623% range across the measured full schemes.	Sensitivity is high but still only a diagnostic, not a security proof.
Finite precision orbit collapse	Exact double logistic orbits showed no repeat within 10^6 steps across 16 seeds. The 16-bit quantized logistic stress test repeated in all 16 seeds after 35–549 steps (median 161; cycle length 5–245, median 60). The current fixed-point tent probe also showed no repeat within 10^6 steps.	Finite precision clearly changes effective orbit length; the exact implementation still needs a longer-horizon orbit-length or state-space study before any stronger claim.

standard stream ciphers intentionally do not provide plaintext avalanche. The candidate pipelines were not subjected to the same security harness and contain fixed prototype permutations. No cryptographic-security conclusion is made for them.

5.9 Negative-Control Summary

The control probes below are the ones reviewers usually ask for when a chaos paper leans too hard on image statistics. They are useful precisely because they do not prove security: they expose misuse sensitivity, limited avalanche under keystream reuse, and the missing finite-precision analysis.

The finite-precision row is deliberate: it now records a reduced-precision stress test instead of leaving the analysis blank. The exact double orbit still only gives a lower bound on repeat length, which is the correct level of claim for the current harness.

6 Discussion

6.1 What SIMD Changes

The experiments show that SIMD is not a property of a pipeline label. It is a property of the dependency graph and memory-access pattern of each stage. Regular, independent byte or word operations benefit from AVX2. Global recurrences, transcendental scalar maps, cycle walking, and sort-based permutation restrict the gain.

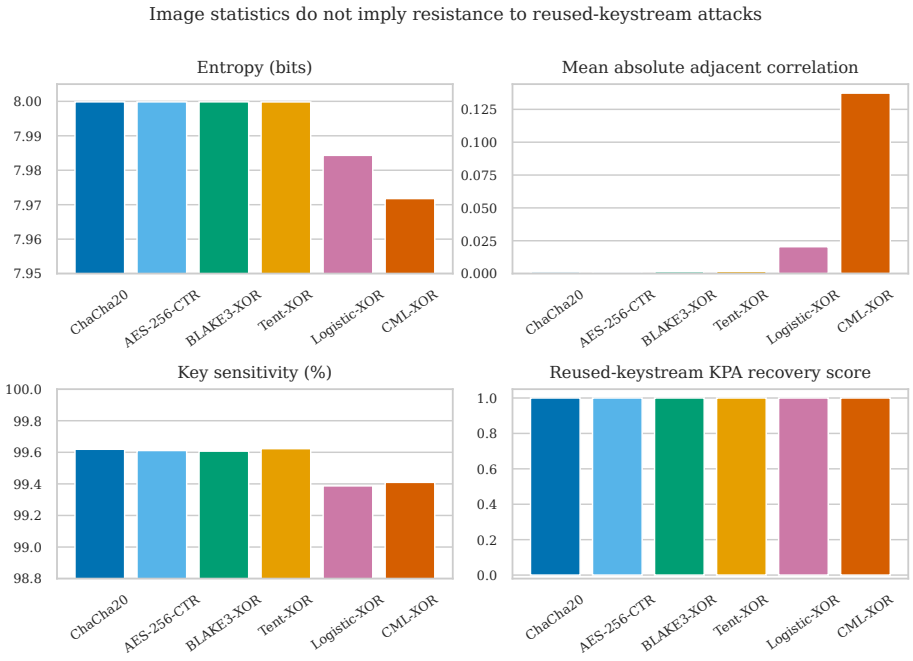

Fig. 6 Selected security diagnostics averaged over the full-scheme experiments. Near-ideal entropy, correlation, and key sensitivity coexist with complete recovery in the deliberately reused-keystream known-plaintext probe.

The largest improvement does not come from a vector instruction alone. It comes from replacing chaotic sort with a linear regular mapping. Similarly, the diffusion improvement comes from changing one global chain into multiple lanes or a tree. Algorithmic restructuring precedes vectorization.

6.2 Performance Versus Security

The fastest stage is not necessarily the strongest stage. Fixed block and checkerboard permutations are useful controls for measuring the cost of regular memory movement, but they provide no secret permutation by themselves. The CA generator is fast, but its rule-30-like output has not been established as a cryptographically secure pseudorandom stream. Tree XOR and linear multi-lane diffusion are also structurally analyzable.

A deployment-oriented design should use a standard cryptographic primitive to derive independent subkeys and masks, make every permutation key-derived, include unique nonces, and provide authentication. A practical research direction is to retain image-domain transforms only as a format-aware preprocessing or selective-encryption layer, then protect the result with an authenticated standard cipher.

6.3 Implications for Future Studies

Future evaluations should report isolated stage execution time; include AES and ChaCha through optimized libraries; compare algorithmic complexity before attributing speedup to SIMD; test real images, controlled synthetic inputs, and public video-frame workloads; report repeated measurements with confidence intervals and p95 frame latency; separate image statistics from cryptographic claims; and include negative controls for nonce reuse, known plaintext, and chosen plaintext.

7 Threats to Validity and Limitations

Most measurements were collected on one older x86-64 server CPU and one ARM64 cloud host, and may differ on newer AVX2/AVX-512 processors, ARM NEON/SVE systems, GPUs, or embedded devices. The current JRTIP extension includes a matched x86/ARM real-time video-frame run, but a stronger submission package would add a modern x86 host and more detailed platform controls. CPU frequency, virtualization noise, memory allocation, and cache state were not controlled through hardware performance counters or processor pinning. The benchmark reports wall-clock stage time and throughput, but not cycles per byte, energy, cache misses, branch misses, or instruction counts.

The prefix-XOR helper is only partially vectorized because its intra-vector scan is scalar. BLAKE3 is compiled without architecture-specific SIMD dispatch. Fixed block and checkerboard permutations are performance templates, not secure key-derived permutations. Candidate pipelines do not yet implement inverse transforms or undergo the full cryptanalysis suite. Image-domain diagnostics are reported only for full schemes and do not constitute proofs of security. AES-CTR and ChaCha20 are confidentiality-only baselines; deployment should use authenticated encryption. Finite-precision state-collapse is now covered only as a reduced-precision stress test, and a longer-horizon exact orbit-length study is still needed before any stronger claim about the chaotic maps themselves.

These limitations constrain the claim to stage-level performance analysis and prototype composition. They do not invalidate the finding that sort permutation and global dependencies are major implementation bottlenecks.

8 Reproducibility

The public research artifact is maintained at https://github.com/ravikanthreddy89/chaotic-encryption-survey. The repository includes the benchmark implementation, dataset generators, Kodak downloader, experiment harness, raw outputs, aggregate CSV files, figure-generation scripts, and manuscript source. A clean checkout can be built and the final experiment reproduced with:

```
git clone <repository-url>
cd chaotic-encryption-survey
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Release
```

```
cmake --build build -j
REPS=10 REAL_REPS=10 \
SIZES="512 1024 2048 4096" \
./scripts/run_paper_ready_experiments.sh
```

Primary result files include `results/final/bench.csv`, `stage_bench.csv`, `candidate_schemes.csv`, `analysis.csv`, `dataset_manifest.csv`, and their aggregate statistics. The repository README documents dependencies, quick-start commands, plot generation, and manuscript compilation. The complete source revision used by each experiment is retained in the generated metadata.

The JRTIP real-time extension is reproduced with `RUN_VIDEO=1` and separate output directories for the x86-64 and ARM64 hosts. The committed result-data directories are `results/jrtip_realtime` and `results/jrtip_realtime_arm`; they contain CSV and Markdown measurements only, not generated ciphertext images or downloaded datasets.

9 Conclusion

This work decomposed chaotic image-encryption pipelines into replaceable keystream, permutation, and diffusion stages and evaluated traditional and SIMD-oriented alternatives under a real-time-oriented measurement model. Implementation-hostile components, especially chaotic sort and global feedback dependencies, determine encryption latency and throughput more strongly than the mere use of SIMD instructions.

Linear block permutation improves throughput by more than two orders of magnitude relative to chaotic sort. AVX2 multi-lane and tree diffusion approximately double throughput relative to a global chain. These improvements remain visible in composed candidate pipelines and at resolutions up to 4096×4096. Nevertheless, optimized ChaCha20 and AES-256-CTR remain faster and have substantially stronger security foundations.

For a real-time image-processing venue, the central implementation lesson is that stage-level timing must be converted into frame-level feasibility: milliseconds per frame, FPS-equivalent throughput, p95 video-frame latency, and pass/fail status against 30/60 FPS budgets. The current results include a public 720p/1080p video-frame workload on both x86-64 and ARM64. They show that Checkerboard-CA-ARX meets 30 FPS at both resolutions on both measured hosts, while optimized AES-GCM and ChaCha20 remain faster and meet both 30 FPS and 60 FPS thresholds in the measured video-frame cases.

The security experiments reinforce an equally important conclusion: image-domain statistics must not be presented as cryptographic proof. The current XOR-based full schemes show favorable entropy and key sensitivity but fail differential-avalanche and reused-keystream recovery controls. The benchmark-informed candidates are therefore performance prototypes, not secure ciphers.

Declarations

Funding

No external funding was received for this study.

Conflict of interest

The author declares no competing interests.

Ethics approval and consent to participate

Not applicable. The study uses public image/video benchmark data and synthetic images; it does not involve human participants, animals, or private personal data.

Consent for publication

Not applicable.

Data availability

The benchmark source code, experiment scripts, raw measurements, aggregate tables, manuscript source, and reproduction instructions are available at https://github.com/ravikanthreddy89/chaotic-encryption-survey. The committed JRTIP result directories contain CSV and Markdown measurements only; generated ciphertext images and downloaded datasets are excluded from version control.

Code availability

The benchmark implementation and scripts are available in the same public repository.

Author contribution

Ravikanth Reddy Gudipati conceived the study, implemented the benchmark, conducted the experiments, analyzed the results, and prepared the manuscript.

Use of generative AI and AI-assisted technologies

Generative-AI and large-language-model tools were used for editorial and engineering assistance during preparation of this manuscript and research artifact. The assistance included prose reorganization, claim-boundary review, LaTeX-template conversion support, and suggestions for reproducibility documentation and small automation scripts. The author reviewed all generated text and code suggestions, ran or inspected the reported experiments, verified numerical claims against committed result files, and takes full responsibility for the final manuscript and artifact.

References

[1] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949, doi: 10.1002/j.1538-7305.1949.tb00928.x.

[2] J. Fridrich, "Symmetric ciphers based on two-dimensional chaotic maps," *International Journal of Bifurcation and Chaos*, vol. 8, no. 6, pp. 1259–1284, 1998, doi: 10.1142/S021812749800098X.

[3] C. Fu, J.-B. Huang, N.-N. Wang, Q.-B. Hou, and W.-M. Lei, "A symmetric chaos-based image cipher with an improved bit-level permutation strategy," *Entropy*, vol. 16, no. 2, pp. 770–788, 2014, doi: 10.3390/e16020770.

[4] L. Huang, S. Cai, M. Xiao, and X. Xiong, "A simple chaotic map-based image encryption system using both plaintext related permutation and diffusion," *Entropy*, vol. 20, no. 7, Art. no. 535, 2018, doi: 10.3390/e20070535.

[5] X. Chai *et al.*, "A new color image encryption scheme using CML and a fractional-order chaotic system," *PLoS ONE*, vol. 10, no. 3, Art. no. e0119660, 2015, doi: 10.1371/journal.pone.0119660.

[6] W. Zhang, Y. Wang, and K. A. Ross, "Parallel prefix sum with SIMD," arXiv:2312.14874, 2023, doi: 10.48550/arXiv.2312.14874.

[7] Y. Wu, J. P. Noonan, and S. Agaian, "NPCR and UACI randomness tests for image encryption," *Journal of Selected Areas in Telecommunications*, pp. 31–38, 2011.

[8] C. Li, D. Lin, B. Feng, J. Lü, and F. Hao, "Cryptanalysis of a chaotic image encryption algorithm based on information entropy," *IEEE Access*, vol. 6, pp. 75834–75842, 2018, doi: 10.1109/ACCESS.2018.2883690.

[9] D. Jiang, Z. Yuan, W. Li, and L. Lu, "Real-time chaotic video encryption based on multithreaded parallel confusion and diffusion," *arXiv preprint arXiv:2302.07411*, 2023. [Online]. Available: https://arxiv.org/abs/2302.07411

[10] C. S. Sanaboina and T. Yadla, "An effective approach to scramble multiple diagnostic imageries using chaos-based cryptography," *arXiv preprint arXiv:2406.07560*, 2024. [Online]. Available: https://arxiv.org/abs/2406.07560

[11] M. Ali *et al.*, "A chaotic image encryption scheme using novel geometric block permutation and dynamic substitution," *arXiv preprint arXiv:2503.09939*, 2025. [Online]. Available: https://arxiv.org/abs/2503.09939

[12] W. Li *et al.*, "Image encryption via data-identified discrete chaotic maps," *arXiv preprint arXiv:2605.21118*, 2026. [Online]. Available: https://arxiv.org/abs/2605.21118

[13] Y. Liu, L. Y. Zhang, J. Wang, Y. Zhang, and K.-W. Wong, "Chosen-plaintext attack of an image encryption scheme based on modified permutation-diffusion structure," arXiv:1503.06638, 2015, doi: 10.48550/arXiv.1503.06638.

[14] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, updated 2023, doi: 10.6028/NIST.FIPS.197-upd1.

[15] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF protocols," RFC 8439, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8439

[16] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, "BLAKE3: One function, fast everywhere," 2020. [Online]. Available: https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[17] R. Franzen, "Kodak lossless true color image suite." [Online]. Available: https://r0k.us/graphics/kodak/